File System

Filosofia Unix

"Everything is a file" - file regolari, directory, device, socket, pipe.

Gerarchia FHS

`/` root, radice del filesystem
`/bin` comandi essenziali (ls, cp, mv)
`/sbin` comandi di sistema (fsck, init)
`/etc` file configurazione
`/home` directory utenti
`/var` file variabili (log, spool)
`/tmp` file temporanei
`/usr` programmi e dati utente
`/dev` file dispositivi
`/proc` filesystem virtuale (info kernel)

Inode

Struttura dati con metadati file: tipo, permessi, UID/GID, dimensione, timestamp, link count, puntatori a blocchi dati. Ogni file ha un inode unico nel filesystem.

Link

Hard link stesso inode, stesso filesystem

```
ln sorgente dest
```

Soft link punta a pathname, cross-filesystem

```
ln -s sorgente dest
```

Permessi

`r` read (4) leggere file/listare dir
`w` write (2) modificare/creare-eliminare
`x` exec (1) eseguire/accedere dir

Formato: `rwxrwxrwx` = user|group|others

`chmod 755 file` rwxr-xr-x
`chmod u+x file` aggiunge exec a user
`chmod go-w file` rimuove write a group/others
`chown user:group file` cambia proprietario
`umask 022` maschera default (755 per dir)

Bit Speciali

`SUID` (4000) esegue con UID proprietario
`SGID` (2000) esegue con GID gruppo
`Sticky` (1000) solo owner può eliminare (/tmp)

Comandi Shell

Navigazione

`pwd` stampa directory corrente
`cd dir` cambia directory
`cd ..` directory padre
`cd ~` home utente
`cd -` directory precedente

Listare File

`ls` lista file
`ls -l` formato lungo (permessi, size, date)
`ls -a` mostra file nascosti (.*)
`ls -h` dimensioni human-readable
`ls -R` ricorsivo
`ls -t` ordina per data modifica
`ls -S` ordina per dimensione

Gestione File/Directory

`touch file` crea file vuoto / aggiorna timestamp
`mkdir dir` crea directory
`mkdir -p a/b/c` crea path completo
`cp src dst` copia file
`cp -r src dst` copia ricorsiva (directory)
`cp -i src dst` conferma sovrascrittura
`mv src dst` sposta/rinomina
`rm file` elimina file
`rm -r dir` elimina directory ricorsivo
`rm -f file` forza eliminazione
`rmdir dir` elimina directory vuota

Visualizzazione

`cat file` stampa contenuto
`less file` paginatore (q per uscire)
`head -n N file` prime N righe
`tail -n N file` ultime N righe
`tail -f file` segue file in tempo reale

Ricerca

`find dir -name "*.c"` cerca per nome
`find dir -type f` solo file regolari
`find dir -type d` solo directory
`find dir -size +1M` file > 1MB
`find dir -mtime -7` modificati ultimi 7gg
`locate pattern` ricerca nel database
`which cmd` path eseguibile
`whereis cmd` path + man + source

Wildcards

`*` qualsiasi stringa
`?` singolo carattere
`[abc]` uno tra a, b, c
`[a-z]` range di caratteri
`[!abc]` nessuno tra a, b, c

Redirezione e Pipe

`cmd > file` stdout su file (sovrascrive)
`cmd >> file` stdout su file (append)
`cmd < file` stdin da file
`cmd 2> file` stderr su file
`cmd &> file` stdout+stderr su file
`cmd1 | cmd2` pipe: stdout1 → stdin2

Archivi

`tar -cvf arch.tar dir` crea archivio
`tar -xvf arch.tar` estrae archivio
`tar -czvf arch.tar.gz dir` crea + gzip
`tar -xzvf arch.tar.gz` estrae gzip
`gzip file` comprime (file.gz)
`gunzip file.gz` decomprime

Shell Scripting

Base

```
#!/bin/bash          # shebang (prima riga)
chmod +x script.sh   # rende eseguibile
./script.sh          # esegue
```

Variabili

`VAR=valore` assegnazione (no spazi!)
`$VAR` espansione variabile
`${VAR}` espansione esplicita
`export VAR` esporta in ambiente
`readonly VAR` variabile costante
`unset VAR` elimina variabile

Variabili Speciali

`$0` nome dello script
`$1..$9` argomenti posizionali
`$#` numero argomenti
`$@` tutti args (separati)
`$*` tutti args (unica stringa)
`$?` exit status ultimo comando
`$$` PID dello script
`$!` PID ultimo processo background

Quoting

`"..."` espande variabili, preserva spazi
`'...'` letterale, nessuna espansione
`` `cmd` `` command substitution (old)
`$(cmd)` command substitution (new)
`$((expr))` espressione aritmetica

Controllo Argomenti (ESAME)

```
# Pattern standard
if [ $# -ne 2 ]; then
    echo "Uso: $0 <arg1> <arg2>"
    exit 1
fi
DIR="$1"
STR="$2"

# Controllo directory valida
if [ ! -d "$DIR" ]; then
    echo "Errore: $DIR non e' una directory"
    exit 1
fi

# Controllo file esiste
if [ ! -e "$FILE" ]; then
    echo "File non trovato"
    exit 1
fi
```

Test Condizioni

```
if [ condizione ]; then ... fi
[ -e file ]   # esiste
[ -f file ]   # file regolare
[ -d file ]   # directory
[ -r file ]   # leggibile
[ -w file ]   # scrivibile
[ -x file ]   # eseguibile
[ -s file ]   # non vuoto
[ -z "$str" ] # stringa vuota
[ -n "$str" ] # stringa non vuota
[ "$a" = "$b" ]  # uguali
[ "$a" != "$b" ] # diversi
[ $a -eq $b ] # numeri uguali
[ $a -ne $b ] # diversi
[ $a -lt $b ] # minore
[ $a -le $b ] # minore o uguale
[ $a -gt $b ] # maggiore
[ $a -ge $b ] # maggiore o uguale
```

Strutture Controllo

```
# if-then-else
if [ cond ]; then
    cmd
elif [ cond2 ]; then
    cmd
else
    cmd
fi

# case
case $var in
    pattern1) cmd ;;
    pattern2) cmd ;;
    *) default ;;
esac
```

Cicli

```
# for
for var in lista; do
    cmd
done
for i in {1..10}; do echo $i; done
for f in *.txt; do cat "$f"; done

# while
while [ cond ]; do
    cmd
```

```
done

# until (opposto di while)
until [ cond ]; do
    cmd
done
```

Loop su File + grep -q (ESAME)

```
# Conta file .txt contenenti stringa
count=0
for f in "$DIR"/*.txt; do
    if [ -f "$f" ]; then           # e' file regolare?
        if grep -q "$STR" "$f"; then  # -q = silent
            count=$((count + 1))
        fi
    fi
done
echo "Trovati $count file"

# Estrai prima/ultima lettera
BASENAME=$(basename "$FILE")
FIRST=$(echo "$BASENAME" | cut -c1)
LAST=$(echo "$BASENAME" | rev | cut -c1)
# oppure: LAST=${BASENAME: -1}
```

Funzioni

```
nome_funzione() {
    # $1, $2... = argomenti funzione
    local var=valore  # var locale
    return 0  # exit status
}
nome_funzione arg1 arg2
result=$?  # cattura return
```

Input

```
read var           # legge in var
read -p "msg: " v  # con prompt
read -s pass       # silent (password)
read -t 5 var      # timeout 5 sec
```

Espansioni Variabili

`${var:-default}`	default se var vuota/non def
`${var:=default}`	assegna default se vuota
`${var:+alt}`	alt se var definita
`${var:?msg}`	errore se var vuota
`${#var}`	lunghezza stringa
`${var:offset:len}`	substring
`${var#pattern}`	rimuovi prefix (shortest)
`${var##pattern}`	rimuovi prefix (longest)
`${var%pattern}`	rimuovi suffix (shortest)
`${var%%pattern}`	rimuovi suffix (longest)
`${var/old/new}`	sostituisci prima
`${var//old/new}`	sostituisci tutte

```
file="/path/to/script.sh"
echo ${file##*/}     # script.sh (basename)
echo ${file%/*}      # /path/to (dirname)
echo ${file%.sh}     # /path/to/script
echo ${#file}        # 20 (lunghezza)
echo ${file:5:7}     # /to/scr (substring)
```

Text Processing

grep - Ricerca Pattern

`grep pattern file`	cerca pattern
`-i`	case insensitive
`-v`	inverti match (righe senza pattern)
`-c`	conta occorrenze
`-n`	mostra numeri riga
`-l`	solo nomi file con match
`-r`	ricorsivo
`-E`	extended regex (ERE)
`-o`	solo parte matchata
`-w`	match parola intera
`-A n`	mostra n righe dopo match
`-B n`	mostra n righe prima match
`-C n`	mostra n righe contesto

Esempi grep

```
# Righe che iniziano con #
grep '^#' file.txt

# Righe vuote
grep '^$' file.txt

# Parola intera "error"
grep -w 'error' log.txt

# Email (pattern semplice)
grep -E '[a-z]+@[a-z]+\.[a-z]+' file

# Numeri interi
grep -E '[0-9]+' file

# Linee NON vuote e NON commenti
grep -v '^$' file | grep -v '^#'

# IP address (pattern base)
grep -E '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' log

# Cerca in tutti i .c ricorsivamente
grep -r 'main' --include='*.c' .

# Conta occorrenze per file
grep -c 'TODO' *.c

# Solo nomi file con match
grep -l 'ERROR' *.log
```

Regex Base (BRE)

`.`	qualsiasi carattere
`*`	0+ occorrenze precedente
`^`	inizio riga
`$`	fine riga
`[abc]`	uno tra a,b,c
`[^abc]`	nessuno tra a,b,c
`[a-z]`	range caratteri
`\<`	inizio parola
`\>`	fine parola

Regex Estese (ERE) - grep -E / egrep

`+`	1+ occorrenze
`?`	0 o 1 occorrenza
`{n}`	esattamente n volte
`{n,m}`	da n a m volte
`|`	alternativa (or)
`(...)`	gruppo

Classi POSIX

`[:alpha:]`	lettere
`[:digit:]`	cifre
`[:alnum:]`	lettere + cifre
`[:space:]`	spazi
`[:upper:]`	maiuscole
`[:lower:]`	minuscole

sed - Stream Editor

```
sed 's/old/new/' file      # sostituisce prima occorrenza
sed 's/old/new/g' file     # sostituisce tutte
sed 's/old/new/gi' file    # case insensitive
sed -i 's/old/new/g' file # modifica in-place
sed -n 'Np' file           # stampa riga N
sed -n '1,10p' file        # stampa righe 1-10
sed 'Nd' file              # elimina riga N
sed '/pattern/d' file      # elimina righe con pattern
sed 's/.*/\U&/' file       # uppercase
sed 's/.*/\L&/' file       # lowercase
```

Backreference sed

```
sed 's/\(.*\)/[\1]/' file  # cattura gruppo con \(...\)
sed 's/\([0-9]*\)/NUM:\1/' # \1 = primo gruppo catturato
& = intera stringa matchata
```

awk - Pattern Processing

```
awk '{ print }' file           # stampa tutto
awk '{ print $1 }' file        # prima colonna
awk '{ print $1, $3 }' file    # colonne 1 e 3
awk '{ print $NF }' file       # ultima colonna
awk -F: '{ print $1 }' /etc/passwd  # separatore :
```

Variabili awk

`$0`	intera riga
`$1..$N`	campo N
`NR`	numero riga corrente
`NF`	numero campi riga
`FS`	field separator (input)
`OFS`	output field separator
`RS`	record separator

Struttura awk

```
awk 'BEGIN { init } pattern { action } END { cleanup }'
# Esempi:
awk 'NR > 1' file              # salta header
awk '$3 > 100' file            # filtro su colonna
awk '{ sum += $1 } END { print sum }' # somma colonna
awk '{ count[$1]++ } END { for(k in count) print k, count[k] }'
```

Altri Comandi

`cut -d: -f1`	estrae campo 1 (delim :)
`cut -c1-10`	caratteri 1-10
`sort`	ordina righe
`sort -n`	ordine numerico
`sort -r`	ordine inverso
`sort -k2`	ordina per colonna 2
`sort -t: -k3 -n`	ordina per campo 3, delim :
`uniq`	rimuove duplicati consecutivi
`uniq -c`	conta occorrenze
`wc -l`	conta righe
`wc -w`	conta parole
`wc -c`	conta byte
`tr 'a-z' 'A-Z'`	traduce caratteri
`tr -d 'chars'`	elimina caratteri

Pattern Esame - Pipeline

```
# Top N processi per thread (decrescente)
ps -eo pid,nlwp,comm | sort -k2 -nr | head -3

# PID con massimo numero thread
ps -eo pid,nlwp | sort -k2 -nr | awk 'NR==1{max=$2} $2==max{print $1}'

# File piu' grande + sostituisci (X) con x
ls -l | grep '^-' | sort -k5 -nr | head -1 | awk '{print $NF}' | \
  xargs cat | sed 's/([^)]*)/(x)/g'

# Conta processi per utente
ps -eo user | sort | uniq -c | sort -nr

# Estrai campo da output
awk -F: '{print $1}' /etc/passwd
cut -d: -f1 /etc/passwd
```

awk Avanzato

```
# Trova massimo e stampa righe con quel valore
awk 'NR==1 {max=$2} $2>max {max=$2} END {...}'
awk 'NR==1 {max=$2} $2==max {print $1}'  # dopo sort -nr

# Array associativi
awk '{count[$1]++} END {for(k in count) print k, count[k]}'

# Condizioni
awk '$3 > 100 && $4 != "root" {print $1}'
awk '/pattern/ {print $2}'    # righe con pattern
awk '!/pattern/ {print}'      # righe senza pattern

# Calcoli
awk '{sum+=$1; n++} END {print sum/n}'  # media
awk 'BEGIN{max=0} $1>max{max=$1} END{print max}'
```

sed Avanzato

```
# Sostituisci contenuto parentesi
sed 's/([^)]*)/(x)/g'      # (abc) -> (x)
sed 's/\(.*\)/[\1]/'       # abc -> [abc] (backreference)

# Elimina righe
sed '/^#/d'       # commenti
sed '/^$/d'       # righe vuote
sed '1d'          # prima riga
sed '$d'          # ultima riga

# Intervalli
sed -n '5,10p'    # righe 5-10
sed '5,10d'       # elimina 5-10
```

```
sed -n '/start/,/end/p'   # tra pattern

# Multiple sostituzioni
sed -e 's/a/A/' -e 's/b/B/'

# Inserisci/Aggiungi
sed '3i\testo'   # inserisci prima riga 3
sed '3a\testo'   # aggiungi dopo riga 3
sed '/pattern/a\nuova riga'  # dopo match

# Sostituzioni con backreference
sed 's/\([0-9]*\)-\([0-9]*\)/\2-\1/'  # swap: 12-34 -> 34-12
sed 's/^\(.*\)$/(\1)/'     # wrappa in parentesi

# In-place con backup
sed -i.bak 's/old/new/g' file

# Rimuovi spazi bianchi
sed 's/^[ \t]*//'    # leading
sed 's/[ \t]*$//'    # trailing
sed 's/^[ \t]*//;s/[ \t]*$//'  # entrambi
```

awk Esempi Pratici

```
# Stampa righe lunghe piu' di 80 char
awk 'length > 80'

# Stampa righe tra due pattern
awk '/START/,/END/'

# Somma colonna 2 dove colonna 1 = "valore"
awk '$1=="valore" {sum+=$2} END {print sum}'

# Trova duplicati
awk 'seen[$0]++' file       # stampa duplicati
awk '!seen[$0]++' file      # rimuovi duplicati

# Rinumera righe
awk '{print NR": "$0}'

# Campo separato da virgola (CSV)
awk -F',' '{print $1, $3}' file.csv

# Sostituisci campo
awk '{$2="nuovo"; print}' OFS='\t'

# Formattazione output
awk '{printf "%-10s %5d\n", $1, $2}'

# Split in array
awk '{split($1, arr, ":"); print arr[1]}'

# Lunghezza stringa
awk '{print $1, length($1)}'

# Substring
awk '{print substr($1, 1, 5)}'  # primi 5 char

# Conversione case
awk '{print toupper($1), tolower($2)}'
```

awk Scripting

```
#!/usr/bin/awk -f
# file: script.awk - eseguire con: awk -f script.awk input

BEGIN {
    FS = ":"            # separatore input
    OFS = "\t"          # separatore output
    count = 0
    print "=== INIZIO ==="
}

# Pattern: esegui solo su righe che matchano
/^[^#]/ {               # ignora commenti
    count++
    data[$1] = $2       # array associativo
}

# Condizione numerica
$3 > 100 {
    print "ALTO:", $1, $3
}

END {
    print "=== FINE ==="
    print "Processate", count, "righe"

    # Itera su array
    for (key in data) {
        print key, "->", data[key]
    }
}
```

Controllo Flusso awk

```
# if-else
awk '{
    if ($1 > 10)
        print "grande"
    else if ($1 > 5)
        print "medio"
    else
        print "piccolo"
}'

# for loop
awk '{
    for (i = 1; i <= NF; i++)
        print "Campo", i, "=", $i
}'

# while loop
awk '{
    i = 1
    while (i <= NF) {
        print $i
        i++
    }
}'

# next (salta a prossima riga)
awk '/skip/ {next} {print}'

# exit (termina)
awk '{if ($1 == "STOP") exit; print}'
```

Funzioni awk

```
# Funzioni stringa
length(s)           # lunghezza
substr(s, start, n) # substring
index(s, t)         # posizione di t in s
split(s, arr, sep)  # split in array
gsub(regex, repl, s) # sostituisci tutte
sub(regex, repl, s)  # sostituisci prima
tolower(s) / toupper(s)
sprintf(fmt, ...)   # formatta stringa

# Funzioni matematiche
int(x)   sin(x)   cos(x)   sqrt(x)
log(x)   exp(x)   rand()   srand()

# Funzione custom
awk '
function somma(a, b) {
    return a + b
}
{print somma($1, $2)}
'

# Esempio pratico: calcola statistiche
awk '
BEGIN {min = 999999; max = 0; sum = 0}
{
    if ($1 < min) min = $1
    if ($1 > max) max = $1
    sum += $1
    n++
}
END {
    print "Min:", min
    print "Max:", max
    print "Media:", sum/n
}' file
```

Sviluppo C

Fasi Compilazione GCC

1. **Preprocessor:** espande #include, #define → .i
2. **Compiler:** C → Assembly → .s
3. **Assembler:** Assembly → Object code → .o
4. **Linker:** unisce .o + librerie → eseguibile

Flag GCC

`-o nome`	nome output
`-c`	solo compile (produce .o)
`-Wall`	abilita warning comuni
`-Wextra`	warning extra
`-Werror`	warning → errori
`-g`	info debug per GDB
`-O0`	no ottimizzazione
`-O1/-O2/-O3`	livelli ottimizzazione
`-I dir`	path include headers
`-L dir`	path librerie
`-l lib`	linka libreria (es: -lm, -lpthread)
`-E`	solo preprocessing
`-S`	produce assembly

```
gcc -Wall -Wextra -g -o prog main.c utils.c -lm
gcc -c main.c       # produce main.o
gcc main.o utils.o -o prog  # linking
```

Formato ELF

Executable and Linkable Format - standard Linux per binari.

Struttura:

- ELF Header: architettura, entry point
- Program Header: segmenti per loader
- Section Header: sezioni per linker

```
readelf -h prog   # mostra header
readelf -S prog   # mostra sezioni
ldd prog          # librerie dinamiche
```

Layout Memoria Processo

Text codice (read-only, eseguibile)
Data variabili globali inizializzate
BSS var globali non inizializzate (zero)
Heap allocazione dinamica (↑)
Stack var locali, return addr (↓)

Makefile

```
# Variabili
CC = gcc
CFLAGS = -Wall -g
TARGET = prog
OBJS = main.o utils.o

# Regole: target: dipendenze
$(TARGET): $(OBJS)
  $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
  $(CC) $(CFLAGS) -c $<

clean:
  rm -f $(OBJS) $(TARGET)

.PHONY: clean
```

Variabili automatiche:
- `$@` target
- `$<` prima dipendenza
- `$^` tutte le dipendenze
- `$*` stem (pattern matching)

System Call vs Libreria

System Call interfaccia diretta kernel
open, read, write, fork, exec
Libreria wrapper + funzionalità
fopen, printf, malloc

I/O System Calls

File Descriptor

Intero non-negativo che identifica file aperto nel processo.
0 STDIN_FILENO (stdin)
1 STDOUT_FILENO (stdout)
2 STDERR_FILENO (stderr)

open / creat

```c
#include <fcntl.h>
int open(const char *path, int oflag);
int open(const char *path, int oflag, mode_t mode);
int creat(const char *path, mode_t mode);
// ret: fd on success, -1 on error
```

Flag obbligatori (uno solo):
- `O_RDONLY` solo lettura
- `O_WRONLY` solo scrittura
- `O_RDWR` lettura + scrittura

Flag opzionali (combinabili con |):
- `O_CREAT` crea se non esiste (richiede mode)
- `O_EXCL` errore se esiste (con O_CREAT)
- `O_TRUNC` tronca a 0 se esiste
- `O_APPEND` scrivi sempre in fondo
- `O_NONBLOCK` I/O non bloccante

```c
int fd = open("file.txt", O_RDWR | O_CREAT, 0644);
```

close

```c
#include <unistd.h>
int close(int fd);  // ret: 0 ok, -1 error
```

read / write

```c
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
// ret: byte letti/scritti, 0 = EOF, -1 = error
```

```c
char buf[1024];
ssize_t n = read(fd, buf, sizeof(buf));
if (n > 0) write(STDOUT_FILENO, buf, n);
```

lseek

```c
#include <unistd.h>
off_t lseek(int fd, off_t offset, int whence);
// ret: nuova posizione, -1 on error
```

- `SEEK_SET` offset da inizio file
- `SEEK_CUR` offset da posizione corrente
- `SEEK_END` offset da fine file

```c
lseek(fd, 0, SEEK_SET);   // vai a inizio
lseek(fd, 0, SEEK_END);   // vai a fine
lseek(fd, -10, SEEK_CUR); // indietro 10 byte
off_t size = lseek(fd, 0, SEEK_END);  // dimensione file
```

dup / dup2

```c
#include <unistd.h>
int dup(int oldfd);               // ret: nuovo fd
int dup2(int oldfd, int newfd);   // chiude newfd, duplica
```

```c
// Redirige stdout su file
int fd = open("out.txt", O_WRONLY | O_CREAT, 0644);
dup2(fd, STDOUT_FILENO);  // 1 ora punta a out.txt
close(fd);
printf("va su file\n");   // scrive su out.txt
```

stat / fstat / lstat

```c
#include <sys/stat.h>
int stat(const char *path, struct stat *buf);
int fstat(int fd, struct stat *buf);
int lstat(const char *path, struct stat *buf); // no follow symlink
```

struct stat (campi principali):
- `st_mode` tipo file + permessi
- `st_size` dimensione in byte
- `st_uid/st_gid` owner UID/GID
- `st_nlink` numero hard link
- `st_mtime` ultima modifica

Macro test tipo (su st_mode):
- `S_ISREG(m)` file regolare
- `S_ISDIR(m)` directory
- `S_ISLNK(m)` symbolic link
- `S_ISFIFO(m)` FIFO/pipe
- `S_ISSOCK(m)` socket

Altre Syscall File

```c
int access(const char *path, int mode); // R_OK,W_OK,X_OK,F_OK
int chmod(const char *path, mode_t mode);
int chown(const char *path, uid_t owner, gid_t group);
int link(const char *old, const char *new);   // hard link
int symlink(const char *old, const char *new); // soft link
int unlink(const char *path);  // elimina file
int rename(const char *old, const char *new);
int mkdir(const char *path, mode_t mode);
int rmdir(const char *path);
```

Processi

Identificatori

```c
#include <unistd.h>
pid_t getpid(void);   // PID processo corrente
pid_t getppid(void);  // PID processo padre
uid_t getuid(void);   // Real UID
uid_t geteuid(void);  // Effective UID
gid_t getgid(void);   // Real GID
gid_t getegid(void);  // Effective GID
```

Real vs Effective:
- Real: utente che ha lanciato il processo
- Effective: determina permessi effettivi (SUID)

Layout Memoria

Text codice eseguibile (RO, condivisibile)
Data globali/statiche inizializzate
BSS globali/statiche non init (zero)
Heap malloc/calloc/realloc (↑)
Stack variabili locali, return addr (↓)

Ambiente

```c
extern char **environ;  // array variabili ambiente
char *getenv(const char *name);
int putenv(char *string);  // "NAME=value"
int setenv(const char *name, const char *val, int overwrite);
int unsetenv(const char *name);
```

fork

```c
#include <unistd.h>
pid_t fork(void);
// ret: 0 nel figlio, PID figlio nel padre, -1 errore
```

```c
pid_t pid = fork();
if (pid < 0) {
    perror("fork"); exit(1);
} else if (pid == 0) {
    // FIGLIO
    printf("Sono figlio, PID=%d\n", getpid());
} else {
    // PADRE
    printf("Sono padre, figlio PID=%d\n", pid);
}
```

Il figlio eredita: codice, dati (copia), fd aperti, ambiente.
Non eredita: PID, PPID, lock, timer.

exec - Famiglia

Sostituisce immagine processo con nuovo programma.

```c
#include <unistd.h>
int execl(const char *path, const char *arg0, ..., NULL);
int execlp(const char *file, const char *arg0, ..., NULL);
int execle(const char *path, const char *arg0, ..., NULL, char *const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
// ret: -1 solo su errore (non ritorna se successo)
```

- `l` lista argomenti (arg0, arg1, ..., NULL)
- `v` vettore argv[]
- `p` cerca in PATH
- `e` passa ambiente envp[]

```c
execl("/bin/ls", "ls", "-l", NULL);
execlp("ls", "ls", "-la", NULL);
char *args[] = {"ls", "-l", NULL};
execvp("ls", args);
```

exit / _exit

```c
#include <stdlib.h>
void exit(int status);   // flush buffer, cleanup
#include <unistd.h>
void _exit(int status);  // termina subito (no cleanup)
```

wait / waitpid

```c
#include <sys/wait.h>
pid_t wait(int *status);  // attende qualsiasi figlio
pid_t waitpid(pid_t pid, int *status, int options);
// ret: PID figlio terminato, -1 errore
```

waitpid pid:
- `> 0` attende figlio specifico
- `-1` qualsiasi figlio
- `0` qualsiasi figlio stesso group

Options: `WNOHANG` (non blocca), `WUNTRACED`

Macro su status:
- `WIFEXITED(s)` true se exit normale
- `WEXITSTATUS(s)` exit code (0-255)
- `WIFSIGNALED(s)` true se terminato da segnale
- `WTERMSIG(s)` numero segnale

```c
int status;
pid_t pid = wait(&status);
if (WIFEXITED(status))
    printf("Exit code: %d\n", WEXITSTATUS(status));
```

Zombie e Orfani

Zombie figlio terminato, padre non ha fatto wait
Orfano padre terminato, figlio adottato da init

Pattern fork-exec

```c
pid_t pid = fork();
if (pid == 0) {
    // Figlio: esegue nuovo programma
    execvp(args[0], args);
    perror("exec"); _exit(1);
}
// Padre: attende
int status;
waitpid(pid, &status, 0);
```

system

```c
#include <stdlib.h>
int system(const char *command);
// ret: exit status comando, -1 errore fork, 127 errore exec
// Internamente: fork + exec("/bin/sh", "-c", cmd) + wait
```

vfork (ottimizzazione)

```c
pid_t pid = vfork();
if (pid == 0) {
    exec...();   // SOLO exec o _exit!
    _exit(1);    // MAI modificare var o return!
}
// Padre bloccato finche' figlio fa exec/_exit
```

waitpid WNOHANG (polling)

```c
// Non bloccante: controlla senza aspettare
int status;
pid_t ret = waitpid(-1, &status, WNOHANG);
if (ret == 0) {
    // nessun figlio terminato ancora
} else if (ret > 0) {
    // figlio ret terminato
    if (WIFEXITED(status))
        printf("Exit: %d\n", WEXITSTATUS(status));
} else {
    // errore o no figli
}
```

Segnali

Segnali Comuni

Num	Nome	Descrizione
1	SIGHUP	hangup terminale
2	SIGINT	Ctrl+C
3	SIGQUIT	Ctrl+\ (core dump)
9	SIGKILL	termina (non catturabile)
11	SIGSEGV	segmentation fault
13	SIGPIPE	pipe senza lettore
14	SIGALRM	timer alarm scaduto
15	SIGTERM	terminazione (default kill)
17	SIGCHLD	figlio terminato/stopped
18	SIGCONT	continua se stopped
19	SIGSTOP	stop (non catturabile)
20	SIGTSTP	Ctrl+Z
10	SIGUSR1	user-defined 1
12	SIGUSR2	user-defined 2

signal

```c
#include <signal.h>
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
// ret: handler precedente, SIG_ERR on error
```

Handler speciali:
- `SIG_DFL` azione default
- `SIG_IGN` ignora segnale

```c
void handler(int sig) {
    printf("Ricevuto segnale %d\n", sig);
```

```
}
signal(SIGINT, handler);   // cattura Ctrl+C
signal(SIGINT, SIG_IGN);   // ignora Ctrl+C
signal(SIGINT, SIG_DFL);   // ripristina default
```

kill / raise

```
#include <signal.h>
int kill(pid_t pid, int sig);  // invia a processo
int raise(int sig);            // invia a se stesso
// ret: 0 ok, -1 error
```

kill pid:
- `> 0` processo specifico
- `0` tutti processi stesso gruppo
- `-1` tutti processi (permessi)
- `< -1` gruppo |pid|

alarm / pause

```
#include <unistd.h>
unsigned int alarm(unsigned int seconds);
// ret: secondi rimanenti alarm precedente
int pause(void);  // sospende fino a segnale
```

```
void timeout(int sig) { printf("Timeout!\n"); }
signal(SIGALRM, timeout);
alarm(5);   // SIGALRM tra 5 sec
pause();    // attende segnale
```

Insiemi di Segnali

```
#include <signal.h>
sigset_t set;
int sigemptyset(sigset_t *set);  // inizializza vuoto
int sigfillset(sigset_t *set);   // tutti i segnali
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
int sigismember(const sigset_t *set, int signum);
```

sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

- `SIG_BLOCK` aggiungi set alla maschera
- `SIG_UNBLOCK` rimuovi set dalla maschera
- `SIG_SETMASK` sostituisci maschera con set

```
sigset_t mask, oldmask;
sigemptyset(&mask);
sigaddset(&mask, SIGINT);
sigprocmask(SIG_BLOCK, &mask, &oldmask);  // blocca SIGINT
// ... sezione critica ...
sigprocmask(SIG_SETMASK, &oldmask, NULL); // ripristina
```

sigaction (raccomandato)

```
#include <signal.h>
struct sigaction {
    void (*sa_handler)(int);          // handler semplice
    void (*sa_sigaction)(int, siginfo_t *, void *); // handler esteso
    sigset_t sa_mask;   // segnali bloccati durante handler
    int sa_flags;       // opzioni
};
int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact);
```

```
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;  // o SA_RESTART
sigaction(SIGINT, &sa, NULL);
```

Flag comuni:
- `SA_RESTART` riavvia syscall interrotte
- `SA_SIGINFO` usa sa_sigaction
- `SA_NOCLDSTOP` no SIGCHLD per stop

alarm + read Timeout (ESAME)

```
volatile sig_atomic_t timeout_flag = 0;

void handler(int s) { timeout_flag = 1; }

int main() {
    char buf[100];
    signal(SIGALRM, handler);

    printf("Hai 5 secondi: ");
    alarm(5);  // avvia timer
    int n = read(0, buf, sizeof(buf));
    alarm(0);  // cancella timer

    if (n < 0 && timeout_flag) {
        printf("Timeout!\n");
    } else {
        printf("Letto: %.*s", n, buf);
    }
}
```

kill(pid, 0) - Test Esistenza

```
if (kill(pid, 0) == 0) {
    printf("Processo %d esiste\n", pid);
} else if (errno == ESRCH) {
    printf("Processo non esiste\n");
} else {
    printf("Nessun permesso\n");
}
```

Gestione SIGPIPE (ESAME)

SIGPIPE inviato quando write su pipe/socket chiuso. Default: termina!

Approccio 1 - Ignora globalmente:

```
signal(SIGPIPE, SIG_IGN);  // ignora
// write ritorna -1, controllo errno:
if (write(fd, buf, n) == -1 && errno == EPIPE)
    printf("peer disconnesso\n");
```

Approccio 2 - Blocca per thread:

```
sigset_t set;
sigemptyset(&set);
sigaddset(&set, SIGPIPE);
pthread_sigmask(SIG_BLOCK, &set, NULL);
// ora write ritorna -1/EPIPE
```

Approccio 3 - MSG_NOSIGNAL (socket):

```
// Solo con send(), non write()!
send(fd, buf, n, MSG_NOSIGNAL);
if (ret == -1 && errno == EPIPE) ...
```

IPC

Pipe (anonime)

Comunicazione unidirezionale tra processi correlati (padre-figlio).

```
#include <unistd.h>
int pipe(int fd[2]);
// fd[0] = lettura, fd[1] = scrittura
// ret: 0 ok, -1 error
```

Pattern padre-figlio:

```
int fd[2];
pipe(fd);
pid_t pid = fork();
if (pid == 0) {
    // FIGLIO: scrive
    close(fd[0]);  // chiude lettura
    write(fd[1], "msg", 3);
    close(fd[1]);
} else {
    // PADRE: legge
    close(fd[1]);  // chiude scrittura
    char buf[100];
    read(fd[0], buf, sizeof(buf));
    close(fd[0]);
}
```

Redirezionare I/O con pipe:

```
// stdout del figlio -> pipe
close(fd[0]);
dup2(fd[1], STDOUT_FILENO);
close(fd[1]);
exec...();
```

FIFO (Named Pipe)

Pipe con nome nel filesystem, per processi non correlati.

```
#include <sys/stat.h>
int mkfifo(const char *pathname, mode_t mode);
// ret: 0 ok, -1 error (EEXIST se esiste)
```

```
mkfifo /tmp/myfifo  # da shell
```

```
// Processo scrittore
int fd = open("/tmp/myfifo", O_WRONLY);  // blocca
write(fd, "data", 4);
close(fd);

// Processo lettore
int fd = open("/tmp/myfifo", O_RDONLY);  // blocca
read(fd, buf, sizeof(buf));
close(fd);
```

Nota: open blocca finche' non c'e' processo all'altro capo (a meno di O_NONBLOCK).

FIFO O_NONBLOCK

- `O_RDONLY` blocca finche' scrittore
- `O_WRONLY` blocca finche' lettore
- `O_RDONLY|O_NONBLOCK` ritorna subito ok
- `O_WRONLY|O_NONBLOCK` errore ENXIO se no lettore

mmap - Memory Mapping

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
int munmap(void *addr, size_t length);
// ret: indirizzo mapping, MAP_FAILED on error
```

Protezione (prot):
- `PROT_READ` leggibile
- `PROT_WRITE` scrivibile
- `PROT_EXEC` eseguibile
- `PROT_NONE` nessun accesso

Flag:
- `MAP_SHARED` modifiche visibili ad altri
- `MAP_PRIVATE` copy-on-write
- `MAP_ANONYMOUS` no file (fd=-1)

mmap per file

```
int fd = open("file", O_RDWR);
struct stat sb;
fstat(fd, &sb);
char *ptr = mmap(NULL, sb.st_size, PROT_READ|PROT_WRITE,
                 MAP_SHARED, fd, 0);
close(fd);  // fd non piu' necessario
// accedi ptr[0..size-1]
munmap(ptr, sb.st_size);
```

mmap per IPC (memoria condivisa anonima)

```
void *ptr = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                 MAP_SHARED|MAP_ANONYMOUS, -1, 0);
if (fork() == 0) {
    // figlio scrive
    strcpy(ptr, "Hello from child");
    exit(0);
}
wait(NULL);
printf("Padre legge: %s\n", (char*)ptr);
munmap(ptr, 4096);
```

POSIX Shared Memory

```
#include <sys/mman.h>
#include <fcntl.h>
int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);
// Compilare con -lrt
```

```
// Creatore
int fd = shm_open("/myshm", O_CREAT|O_RDWR, 0666);
ftruncate(fd, 4096);  // imposta dimensione
void *ptr = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                 MAP_SHARED, fd, 0);
close(fd);
// ... usa ptr ...
munmap(ptr, 4096);
shm_unlink("/myshm");  // rimuove
```

Semafori POSIX

```
#include <semaphore.h>
sem_t *sem_open(const char *name, int oflag, mode_t mode,
                unsigned int value);
int sem_wait(sem_t *sem);    // P: decrementa (blocca se 0)
int sem_post(sem_t *sem);    // V: incrementa
int sem_close(sem_t *sem);
int sem_unlink(const char *name);
// Compilare con -lpthread
```

```
sem_t *sem = sem_open("/mysem", O_CREAT, 0666, 1); // mutex
sem_wait(sem);   // entra sezione critica
// ... sezione critica ...
sem_post(sem);   // esce
sem_close(sem);
sem_unlink("/mysem");
```

Semafori senza nome (thread)

```
sem_t sem;
sem_init(&sem, 0, 1);  // 0=thread, 1=processi
sem_wait(&sem);
sem_post(&sem);
sem_destroy(&sem);
```

select() con Pipe + stdin (ESAME)

```
int p[2];
pipe(p);
int pipe_closed = 0;
```

```c
if (fork() == 0) {  // FIGLIO
    close(p[1]);    // chiude scrittura
    char buf[64];

    while (1) {
        fd_set set;
        FD_ZERO(&set);
        FD_SET(STDIN_FILENO, &set);  // stdin = 0
        if (!pipe_closed)
            FD_SET(p[0], &set);      // pipe read

        int maxfd = p[0];  // p[0] > 0
        if (select(maxfd + 1, &set, NULL, NULL, NULL) <= 0)
            break;

        if (FD_ISSET(p[0], &set)) {
            int n = read(p[0], buf, sizeof(buf));
            if (n == 0) {  // EOF: padre ha chiuso!
                printf("Pipe chiusa\n");
                pipe_closed = 1;
                close(p[0]);
                continue;  // continua con stdin
            }
            if (n < 0) break;
            printf("PIPE: %.*s", n, buf);
        }
        if (FD_ISSET(STDIN_FILENO, &set)) {
            int n = read(0, buf, sizeof(buf));
            if (n <= 0) {
                if (pipe_closed) break;  // tutto chiuso
                continue;
            }
            printf("STDIN: %.*s", n, buf);
        }
    }
    if (!pipe_closed) close(p[0]);
} else {  // PADRE
    close(p[0]);              // IMPORTANTE: chiude lettura
    write(p[1], "ping\n", 5);
    sleep(2);
    close(p[1]);              // chiude -> EOF al figlio
    wait(NULL);
}
```

Gestione EOF pipe:

- `read() == 0`: EOF, l'altro processo ha chiuso
- Padre DEVE chiudere `fd[1]` per segnalare EOF
- Figlio DEVE chiudere `fd[1]` subito (non lo usa)

Thread POSIX

Concetti Base

Thread = flusso esecuzione leggero, condivide: codice, dati, heap, fd.
Ogni thread ha: stack proprio, registri, TID.
Compilare con: `gcc -pthread`

Creazione e Terminazione

```c
#include <pthread.h>
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
void pthread_exit(void *retval);
pthread_t pthread_self(void);  // ret: TID corrente
```

```c
void *worker(void *arg) {
    int id = *(int*)arg;
    printf("Thread %d\n", id);
    return NULL;  // o pthread_exit(NULL);
}
pthread_t tid;
int arg = 42;
pthread_create(&tid, NULL, worker, &arg);
```

Join e Detach

```c
int pthread_join(pthread_t thread, void **retval);
// blocca finche' thread termina, recupera retval
int pthread_detach(pthread_t thread);
// thread non joinable, risorse liberate auto
```

```c
void *result;
pthread_join(tid, &result);  // attende thread
// oppure
pthread_detach(tid);  // non attendo, risorse auto-liberate
```

Mutex

```c
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
// oppure:
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);  // non blocca
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

```c
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Condition Variables

```c
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
// rilascia mutex, attende signal, riacquista mutex
int pthread_cond_signal(pthread_cond_t *cond);    // sveglia 1
int pthread_cond_broadcast(pthread_cond_t *cond); // sveglia tutti
int pthread_cond_destroy(pthread_cond_t *cond);
```

Pattern Produttore-Consumatore

```c
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
int buffer[N], count = 0;

void *producer(void *arg) {
    int item = produce();
    pthread_mutex_lock(&mutex);
    while (count == N)  // buffer pieno
        pthread_cond_wait(&not_full, &mutex);
    buffer[count++] = item;
    pthread_cond_signal(&not_empty);
    pthread_mutex_unlock(&mutex);
}

void *consumer(void *arg) {
    pthread_mutex_lock(&mutex);
    while (count == 0)  // buffer vuoto
        pthread_cond_wait(&not_empty, &mutex);
    int item = buffer[--count];
    pthread_cond_signal(&not_full);
    pthread_mutex_unlock(&mutex);
    consume(item);
}
```

Semafori POSIX (Thread)

```c
#include <semaphore.h>
sem_t sem;
int sem_init(sem_t *sem, int pshared, unsigned int value);
// pshared: 0=thread, non-zero=processi
int sem_wait(sem_t *sem);     // P: decr, blocca se 0
int sem_trywait(sem_t *sem); // non blocca
int sem_post(sem_t *sem);     // V: incrementa
int sem_getvalue(sem_t *sem, int *sval);
int sem_destroy(sem_t *sem);
```

```c
sem_t empty, full, mutex;
sem_init(&empty, 0, N);  // slot vuoti
sem_init(&full, 0, 0);   // slot pieni
sem_init(&mutex, 0, 1);  // mutua esclusione

// Produttore
sem_wait(&empty);
sem_wait(&mutex);
// inserisci in buffer
sem_post(&mutex);
sem_post(&full);

// Consumatore
sem_wait(&full);
sem_wait(&mutex);
// preleva da buffer
sem_post(&mutex);
sem_post(&empty);
```

Attributi Thread

```c
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
pthread_attr_setstacksize(&attr, 1024*1024);
pthread_create(&tid, &attr, func, arg);
pthread_attr_destroy(&attr);
```

Pattern Turnazione A-B (ESAME)

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  c = PTHREAD_COND_INITIALIZER;
int turn = 0;   // 0 = A, 1 = B

void* threadA(void* arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&m);
        while (turn != 0)               // WHILE, non IF!
            pthread_cond_wait(&c, &m);
        printf("A\n");
        turn = 1;                       // passa turno a B
        pthread_cond_signal(&c);        // sveglia B
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

void* threadB(void* arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&m);
        while (turn != 1)               // WHILE!
            pthread_cond_wait(&c, &m);
        printf("B\n");
        turn = 0;                       // passa turno a A
        pthread_cond_signal(&c);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

Pattern Cascata T1->T2->T3 (ESAME)

```c
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int stato = 1;  // 1=T1, 2=T2, 3=T3

void* thread1(void* arg) {
    pthread_mutex_lock(&mtx);
    while (stato != 1)
        pthread_cond_wait(&cond, &mtx);
    printf("T1 lavora\n");
    stato = 2;
    pthread_cond_broadcast(&cond);  // sveglia TUTTI
    pthread_mutex_unlock(&mtx);
    return NULL;
}
// thread2 e thread3: stessa struttura con stato 2 e 3

int main() {
    pthread_t t1, t2, t3;
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
    pthread_create(&t3, NULL, thread3, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_mutex_destroy(&mtx);
    pthread_cond_destroy(&cond);
}
```

signal vs broadcast:

- `signal`: sveglia 1 thread (piu' efficiente)
- `broadcast`: sveglia tutti (necessario se piu' thread aspettano condizioni diverse)

Errori Comuni (ESAME)

- **if vs while:** usare sempre `while` con cond_wait!
- **Doppio lock:** lock senza unlock = deadlock
- **Unlock mancante:** dopo sezione critica
- **Signal prima unlock:** ok ma unlock prima e' meglio

Sequenza corretta:

```c
pthread_mutex_lock(&m);
while (!condizione)          // WHILE!
    pthread_cond_wait(&c, &m);
// ... lavoro ...
condizione = nuovo_valore;
pthread_cond_signal(&c);     // o broadcast
pthread_mutex_unlock(&m);
```

Lettori-Scrittori (ESAME)

```c
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rw = PTHREAD_MUTEX_INITIALIZER;
int readers = 0;

void *reader(void *arg) {
    pthread_mutex_lock(&mtx);
    readers++;
    if (readers == 1)           // primo lettore
        pthread_mutex_lock(&rw);  // blocca scrittori
    pthread_mutex_unlock(&mtx);

    // --- LETTURA (piu' lettori insieme) ---

    pthread_mutex_lock(&mtx);
```

```
    readers--;
    if (readers == 0)           // ultimo lettore
        pthread_mutex_unlock(&rw);  // sblocca scrittori
    pthread_mutex_unlock(&mtx);
    return NULL;
}

void *writer(void *arg) {
    pthread_mutex_lock(&rw);    // esclusivo
    // --- SCRITTURA ---
    pthread_mutex_unlock(&rw);
    return NULL;
}
```

Thread-Local Storage TLS

Variabili "globali" ma private per ogni thread.

```
#include <pthread.h>
pthread_key_t key;

// Crea chiave (1 volta, in main)
pthread_key_create(&key, free); // free=destructor

// In ogni thread:
pthread_setspecific(key, malloc(N));  // setta
void *p = pthread_getspecific(key);   // legge

// Cleanup finale
pthread_key_delete(key);
```

Esempio completo:

```
pthread_key_t id_key;

void *worker(void *arg) {
    int *my_id = malloc(sizeof(int));
    *my_id = *(int*)arg;
    pthread_setspecific(id_key, my_id);

    // Ora qualsiasi funzione puo' fare:
    int *id = pthread_getspecific(id_key);
    printf("Sono thread %d\n", *id);
    return NULL;  // destructor libera memoria
}

int main() {
    pthread_key_create(&id_key, free);
    // ... crea thread ...
    pthread_key_delete(id_key);
}
```

Socket

Concetti Base

TCP connection-oriented, affidabile, stream
UDP connectionless, non affidabile, datagram

Strutture Dati

```
#include <netinet/in.h>
struct sockaddr_in {
    sa_family_t    sin_family;  // AF_INET
    in_port_t      sin_port;    // porta (network order)
    struct in_addr sin_addr;    // IP address
};
struct in_addr {
    uint32_t s_addr;  // IP (network order)
};
```

Byte Order

```
#include <arpa/inet.h>
uint16_t htons(uint16_t hostshort);  // host to network short
uint32_t htonl(uint32_t hostlong);   // host to network long
uint16_t ntohs(uint16_t netshort);   // network to host short
uint32_t ntohl(uint32_t netlong);    // network to host long
```

Conversione Indirizzi

```
#include <arpa/inet.h>
int inet_pton(int af, const char *src, void *dst);
// "192.168.1.1" -> binary, ret: 1 ok, 0 invalid, -1 error
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
// binary -> string, ret: dst o NULL
```

socket

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
// ret: fd socket, -1 error
```

domain: AF_INET (IPv4), AF_INET6, AF_UNIX
type: SOCK_STREAM (TCP), SOCK_DGRAM (UDP)
protocol: 0 (default)

Server TCP

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t len);
int listen(int sockfd, int backlog);  // backlog: coda connessioni
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
// ret: nuovo fd per client, -1 error
```

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

struct sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;  // tutte le interfacce

bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
listen(server_fd, 5);

struct sockaddr_in client_addr;
socklen_t client_len = sizeof(client_addr);
int client_fd = accept(server_fd, (struct sockaddr*)&client_addr,
                       &client_len);
// client_fd pronto per read/write
char buf[1024];
read(client_fd, buf, sizeof(buf));
write(client_fd, "OK", 2);
close(client_fd);
close(server_fd);
```

Client TCP

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t len);
// ret: 0 ok, -1 error
```

```
int sock = socket(AF_INET, SOCK_STREAM, 0);

struct sockaddr_in server_addr;
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr));

write(sock, "Hello", 5);
char buf[1024];
read(sock, buf, sizeof(buf));
close(sock);
```

send / recv

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
// flags: 0, MSG_WAITALL, MSG_PEEK, MSG_DONTWAIT
```

UDP

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest, socklen_t addrlen);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src, socklen_t *addrlen);
```

```
// Server UDP
int sock = socket(AF_INET, SOCK_DGRAM, 0);
bind(sock, ...);
recvfrom(sock, buf, sizeof(buf), 0, &client_addr, &client_len);
sendto(sock, reply, len, 0, &client_addr, client_len);

// Client UDP
int sock = socket(AF_INET, SOCK_DGRAM, 0);
sendto(sock, msg, len, 0, &server_addr, sizeof(server_addr));
recvfrom(sock, buf, sizeof(buf), 0, NULL, NULL);
```

Opzioni Socket

```
int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
// SO_REUSEADDR: riusa porta subito dopo close
int opt = 1;
setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

select - I/O Multiplexing

```
#include <sys/select.h>
int select(int nfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
// ret: num fd pronti, 0 timeout, -1 error

fd_set set;
FD_ZERO(&set);          // inizializza vuoto
FD_SET(fd, &set);       // aggiungi fd
FD_CLR(fd, &set);       // rimuovi fd
FD_ISSET(fd, &set);     // test se fd nel set
```

```
fd_set readfds;
FD_ZERO(&readfds);
FD_SET(server_fd, &readfds);
FD_SET(client_fd, &readfds);
int maxfd = (server_fd > client_fd) ? server_fd : client_fd;

select(maxfd + 1, &readfds, NULL, NULL, NULL);  // blocca

if (FD_ISSET(server_fd, &readfds))
    // nuova connessione
if (FD_ISSET(client_fd, &readfds))
    // dati dal client
```

Chat UDP con select (ESAME)

```
int sock = socket(AF_INET, SOCK_DGRAM, 0);
struct sockaddr_in local, peer;
char buf[256];

// Setup bind locale
local.sin_family = AF_INET;
local.sin_port = htons(porta_locale);
local.sin_addr.s_addr = htonl(INADDR_ANY);
bind(sock, (struct sockaddr*)&local, sizeof(local));

// Setup peer
peer.sin_family = AF_INET;
peer.sin_port = htons(porta_peer);
inet_pton(AF_INET, "IP_PEER", &peer.sin_addr);

while (1) {
    fd_set set;
    FD_ZERO(&set);
    FD_SET(STDIN_FILENO, &set);  // stdin
    FD_SET(sock, &set);          // socket

    int maxfd = sock;  // sock > 0
    if (select(maxfd + 1, &set, NULL, NULL, NULL) < 0)
        break;

    // Input utente -> invia a peer
    if (FD_ISSET(STDIN_FILENO, &set)) {
        int n = read(0, buf, sizeof(buf));
        if (n <= 0) break;
        sendto(sock, buf, n, 0,
               (struct sockaddr*)&peer, sizeof(peer));
    }

    // Dati da peer -> stampa
    if (FD_ISSET(sock, &set)) {
        struct sockaddr_in from;
        socklen_t fromlen = sizeof(from);
        int n = recvfrom(sock, buf, sizeof(buf), 0,
                         (struct sockaddr*)&from, &fromlen);
        if (n < 0) break;
        printf("lui dice: %.*s", n, buf);
    }
}
close(sock);
```

Errore Deadlock Mutex+Socket (ESAME)

```
// SBAGLIATO - Deadlock!
void* receiver(void* arg) {
    pthread_mutex_lock(&mtx);   // prende lock
    recvfrom(sockfd, ...);      // BLOCCA tenendo lock!
    pthread_mutex_unlock(&mtx);
}
void* sender(void* arg) {
    pthread_mutex_lock(&mtx);   // MAI -> deadlock
    sendto(sockfd, ...);
    pthread_mutex_unlock(&mtx);
}
```

Soluzioni:

1. NON usare mutex per socket (gia' thread-safe)
2. Usare `select()` prima di recvfrom
3. Socket non bloccanti: `MSG_DONTWAIT`

```
// CORRETTO: select prima di recvfrom
fd_set set;
FD_ZERO(&set);
FD_SET(sockfd, &set);
struct timeval tv = {1, 0};  // timeout 1s
if (select(sockfd+1, &set, NULL, NULL, &tv) > 0) {
    recvfrom(sockfd, buf, ...);
}
```

Server TCP select Multiplexed

```
fd_set master, read_set;
FD_ZERO(&master);
FD_SET(listen_sd, &master);
int max_fd = listen_sd;

while (1) {
    read_set = master;  // COPIA! select modifica
    select(max_fd + 1, &read_set, NULL, NULL, NULL);

    for (int i = 0; i <= max_fd; i++) {
        if (!FD_ISSET(i, &read_set)) continue;

        if (i == listen_sd) {
            // Nuova connessione
            int new_sd = accept(listen_sd, NULL, NULL);
```

```
            FD_SET(new_sd, &master);
            if (new_sd > max_fd) max_fd = new_sd;
        } else {
            // Dati da client
            int n = recv(i, buf, sizeof(buf), 0);
            if (n <= 0) {
                close(i);
                FD_CLR(i, &master);
            } else {
                // elabora buf, rispondi...
                send(i, buf, n, 0);  // echo
            }
        }
    }
}
```

Broadcast UDP

```
int opt = 1;
setsockopt(sock, SOL_SOCKET, SO_BROADCAST, &opt, sizeof(opt));

struct sockaddr_in bcast;
bcast.sin_family = AF_INET;
bcast.sin_port = htons(5000);
inet_pton(AF_INET, "255.255.255.255", &bcast.sin_addr);

sendto(sock, msg, len, 0, (struct sockaddr*)&bcast, sizeof(bcast));
```

Multicast UDP

```
// Receiver: iscrizione gruppo
struct ip_mreq mreq;
mreq.imr_multiaddr.s_addr = inet_addr("239.0.0.1");
mreq.imr_interface.s_addr = htonl(INADDR_ANY);
setsockopt(sock, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof(mreq));

// Sender: imposta TTL
int ttl = 64;
setsockopt(sock, IPPROTO_IP, IP_MULTICAST_TTL, &ttl, sizeof(ttl));
```